NBA SHOT SELECTION AND SHOT PREDICTION

A Machine Learning-Based Analysis of Kobe Bryant's Shot Outcomes

Project Code	**PRCP-1008**
Domain	Sports Analytics / Machine Learning
Dataset	Kobe Bryant shot-selection data
Target Variable	shot_made_flag (1 = made, 0 = missed)
Prepared by	Poreddy Yogananda Reddy

PROJECT REPORT

Report Contents

1. Abstract
2. 1. Introduction
3. 2. Problem Statement
4. 3. Project Objectives
5. 4. Dataset Description
6. 5. Software and Technologies
7. 6. Methodology and Workflow
8. 7. Candidate Models and Evaluation Metrics
9. 8. System Design and Outputs
10. 9. Stakeholder Insights and Practical Use
11. 10. Challenges and Mitigation
12. 11. Limitations
13. 12. Future Enhancements
14. 13. Conclusion
15. 14. Results to Complete After Running the Notebook
16. Appendix A – Execution and submission checklist

Abstract

This project investigates Kobe Bryant's historical shot-selection data using statistical analysis, exploratory data analysis, feature engineering and supervised machine learning. The prediction target is shot_made_flag, where 1 represents a made field goal and 0 represents a missed field goal. The notebook is designed to address three objectives: analyse shot patterns, derive evidence-based insights that may support game strategy, and develop a model that estimates the probability of a shot being successful.

The workflow includes data ingestion, Pandera-based validation, missing-target assessment, outlier investigation, spatial and temporal analysis, leakage-aware model validation, comparison of seven classification algorithms, hyperparameter optimisation, probability calibration, model explainability, error analysis, model comparison and a production-oriented prediction workflow. The notebook also includes drift analysis and MLflow tracking.

Important reporting note: the supplied notebook contains code and explanatory markdown but no executed cell outputs. Therefore, dataset totals, final model identity, accuracy, F1-score, ROC-AUC, log-loss, confidence intervals and specific empirical strategy findings cannot be verified from the supplied file alone. This report documents the implemented methodology and identifies the result fields that must be populated after executing the notebook.

1. Introduction

Basketball shot selection is influenced by several circumstances, including shot location, distance from the basket, shot type, action type, game period and time remaining. Analysing these variables can help describe historical shooting patterns and estimate the likelihood that a shot will be converted.

This project uses historical Kobe Bryant shot records to build a binary classification workflow. The model estimates P(shot_made_flag = 1 | shot circumstances). Its output is a probability rather than a guarantee of the outcome. A shot is an individual event and remains subject to uncertainty and factors not represented in the dataset.

2. Problem Statement

The objective is to analyse the supplied NBA shot dataset and develop a reliable model that predicts whether a recorded shot is made or missed. The work must also help stakeholders understand differences in shooting efficiency across court locations, shot categories and game situations.

Project tasks: (1) prepare a complete data analysis; (2) generate data-supported insights that can inform game strategy and compare candidate models using accuracy and F1-score; and (3) build a shot prediction system that returns a make probability and a made/missed decision.

3. Project Objectives

- Inspect and validate the raw dataset, including missing values, duplicates, data types, ranges and unusual observations.
- Explore shooting outcomes by season, shot type, action, court zone, distance, period and game-clock context.
- Apply statistical tests to assess selected relationships rather than relying only on visual comparisons.
- Engineer spatial, temporal and contextual features that are available at the time of a shot.
- Compare multiple classification algorithms with validation that respects game-level grouping and chronology.
- Optimise selected model hyperparameters, choose a probability calibration method and decision threshold using development data.
- Explain model behaviour, examine prediction errors, assess drift and create reusable prediction artifacts.

4. Dataset Description

The notebook retrieves a CSV containing a shot_made_flag column. It first searches for a suitable local CSV, then attempts to download the project ZIP archive, and finally provides a Google Colab upload fallback. The game_date field is parsed as a datetime.

The target variable is shot_made_flag. Rows with a missing target cannot be used for supervised training or labelled performance evaluation. The notebook sets these rows aside and later applies the trained model to generate predictions for them. The notebook's introductory documentation describes approximately 5,000 unlabelled rows; the exact count and total dataset size should be confirmed from a successful execution.

Relevant fields referenced in the notebook include shot_id, game_id, game_date, game_event_id, loc_x, loc_y, shot_distance, minutes_remaining, seconds_remaining, period, playoffs, season, matchup, opponent, action_type, combined_shot_type, shot_type, shot_zone_basic, shot_zone_area and shot_zone_range.

Scope limitation: the data and model are specific to Kobe Bryant's historical shots, described in the notebook as covering 1996–2016. The model should not be assumed to generalise to other players without additional data and validation.

5. Software and Technologies

Python is the implementation language. Pandas and NumPy support tabular data handling and numerical operations. Matplotlib, Seaborn and Plotly support static and interactive visualisation. SciPy and Statsmodels support statistical tests and regression analysis.

Scikit-learn provides preprocessing, pipelines, model validation, metrics, calibration and permutation importance. XGBoost, LightGBM and CatBoost provide gradient-boosted tree classifiers. SHAP supports model explanation. Optuna performs hyperparameter search. Pandera defines data validation rules. MLflow tracks experiments. Joblib serialises the trained artifact. Evidently is an optional dependency for an HTML drift report.

Installing a package does not by itself mean that every capability has been executed successfully. Optional components, including Evidently and MLflow logging, depend on the runtime environment and successful completion of their respective notebook cells.

6. Methodology and Workflow

6.1 Environment setup

The notebook installs the required Python packages, imports the libraries and fixes a random seed (42) for reproducibility. It records package versions and writes environment information to a requirements file.

6.2 Data ingestion

The data-loading function searches local directories for a CSV containing the target field, downloads and extracts the project archive if needed, and offers a Colab upload fallback.

6.3 Data validation and quality

Pandera defines constraints for unique shot identifiers, non-null game identifiers, plausible dates and coordinate/time ranges, valid periods, binary playoff flags, target values and recognised shot-zone categories. Lazy validation reports multiple failures for inspection. The notebook also calculates data types, missingness, unique counts, duplicate rows, constant columns, game count and date coverage.

6.4 Missing target assessment

Rows with missing shot_made_flag are excluded from labelled training and evaluation. Chi-square tests and Cramér's V are used to examine associations between target missingness and observed fields such as period, season, shot type, shot zone and playoff status. This is exploratory and cannot prove that target missingness is random.

6.5 Outlier assessment

The notebook applies the interquartile-range rule to selected numeric fields and uses Isolation Forest with a stated contamination setting of 0.01. Unusual observations are inspected. The notebook's documented decision is to retain plausible long-distance or back-court attempts because these may represent genuine basketball events rather than data errors.

6.6 Exploratory and statistical analysis

The analysis examines target balance and field-goal percentage by season, shot type, combined shot type, shot zone, shot range, court area, period, playoff status and action type. It also visualises shot-distance, game-clock and period patterns. Statistical procedures include chi-square tests, a two-proportion z-test, logistic regression for shot distance, correlation analysis and variance inflation factors.

6.7 Spatial and temporal analysis

Court-coordinate visualisations are used to inspect shot locations and efficiency. Coordinate checks compare radial distance derived from loc_x and loc_y with shot_distance. Temporal analysis groups outcomes by season, calendar year and month, and examines average shot distance over time.

6.8 Feature engineering

The feature builder creates shot angle, absolute shot angle, a distance-angle interaction, distance buckets, seconds remaining in the period, last-five-seconds and last-minute indicators, overtime flag, season number, home/away indicator and game-phase category. It also retains selected shot/action/zone and opponent categories. The notebook explicitly notes that there is no shot-clock variable; last_5s refers to the game clock, not the shot clock.

6.9 Feature selection and leakage control

The notebook documents removal of redundant latitude/longitude transforms, constant team fields and bookkeeping identifiers from model features. game_id is used as a grouping key, not as a predictive feature. This helps avoid treating identifiers as meaningful predictors.

6.10 Validation design

The most recent approximately 20% of games form a chronological hold-out. Earlier games form the development set. The notebook asserts that no game identifier appears in both sets. StratifiedGroupKFold is used for grouped cross-validation so that shots from one game are not split across training and validation folds. Random, group-based and chronological splits are also compared to demonstrate the impact of evaluation design.

6.11 Model development

Seven candidate classifiers are defined: Logistic Regression, Random Forest, Extra Trees, HistGradientBoosting, XGBoost, LightGBM and CatBoost. Categorical variables are one-hot encoded with unknown-category handling and a minimum-frequency setting; numeric scaling is applied where required. Out-of-fold probabilities are generated on the development data.

6.12 Hyperparameter optimisation

Optuna with a TPE sampler tunes XGBoost and LightGBM. The tuning objective is mean log-loss over grouped three-fold cross-validation inside the development data. The chronological hold-out is not used for tuning.

6.13 Ensemble learning

The notebook evaluates soft voting, which averages probabilities from tuned tree models, and nested group-aware stacking. The stacking design rebuilds base-model predictions within inner grouped folds for each outer fold to reduce information leakage into the meta-model.

6.14 Model selection and threshold

The predefined selection score is (Accuracy + F1) / 2, computed from development out-of-fold predictions. Candidates within 0.003 of the top score are treated as tied, with lower log-loss used as the tie-breaker. The decision threshold is searched over 0.30–0.70 using development out-of-fold probabilities.

6.15 Probability calibration

No calibration, Platt (sigmoid) scaling and isotonic regression are compared by cross-fitting on development out-of-fold predictions. The notebook selects the method with the lowest cross-validated Brier score and requires at least a 0.0003 improvement over no calibration before adopting a calibrator.

6.16 Final evaluation

After the model, calibration method and threshold are fixed using development data, the selected configuration is evaluated on the chronological hold-out. Metrics include accuracy, F1, precision, recall, ROC-AUC, log-loss and Brier score. The notebook also describes game-level bootstrap intervals.

6.17 Explainability and error analysis

SHAP is used with a selected tree model to explain feature contributions; permutation importance and feature-interaction analysis are also included. The notebook clarifies that these methods explain model behaviour, not causal effects. Error analysis examines incorrect predictions and performance patterns.

6.18 Statistical model comparison

Paired bootstrap resampling is performed by game clusters to compare model metrics on the same resampled games. McNemar's exact test compares paired hard predictions. The notebook notes that shot-level independence assumptions can make McNemar p-values optimistic, so game-level bootstrap intervals should receive greater emphasis.

6.19 Nested cross-validation

A nested grouped validation procedure tunes LightGBM inside each outer training fold and evaluates on the untouched outer fold. Logistic Regression is treated as a reference. This is intended to estimate the performance of the tuning procedure with less selection optimism.

6.20 Calibration, shot difficulty and drift

Calibration is assessed with Brier score, log-loss, expected calibration error, calibration slope/intercept and reliability bins. The model's expected make probability is used as a shot-quality measure, with difficulty defined as one minus predicted make probability. Drift is assessed between earlier development games and later hold-out games using PSI and KS/chi-square tests, plus target, prediction and seasonal performance comparisons.

7. Candidate Models and Evaluation Metrics

Candidate models: Logistic Regression; Random Forest; Extra Trees; Histogram-based Gradient Boosting; XGBoost; LightGBM; and CatBoost. The notebook additionally evaluates soft voting and nested stacking approaches.

Accuracy measures the fraction of correct classifications. Precision measures the proportion of predicted makes that were actually made. Recall measures the proportion of actual makes identified by the model. F1-score is the harmonic mean of precision and recall. ROC-AUC measures ranking discrimination across thresholds. Log-loss evaluates the quality of predicted probabilities and penalises confident incorrect predictions. Brier score measures squared probability error.

The model-selection rule deliberately combines accuracy and F1 rather than optimising only one metric. Probability metrics are also retained because the intended system returns a make probability, not only a binary label.

8. System Design and Outputs

The notebook saves a final model artifact named kobe_shot_model.joblib. It also writes predictions for originally unlabelled rows to unlabelled_shot_predictions.csv, containing shot_id, shot_made_probability and shot_made_flag_pred.

The production-oriented section generates shot_service.py with shared feature-building and prediction logic, and api.py with FastAPI health and prediction endpoints. A Streamlit dashboard source is also generated. The notebook reloads the service module and checks that predictions from the service match notebook predictions for a sample of 25 rows. The API and dashboard are generated as project artifacts; their presence in the notebook does not alone establish that they were deployed to a live server.

MLflow is configured with a local SQLite tracking URI and an experiment named kobe-shot-prediction. When successful, it records model names, validation description, selection score, selected hyperparameters, metrics and final artifacts.

9. Stakeholder Insights and Practical Use

The notebook computes shot efficiency by zone and action type, subject to minimum attempt-count filters. It calculates points per shot so that two-point and three-point opportunities can be compared on a common scoring-value basis. It also examines the final 24 seconds of a period and compares actual and expected field-goal percentages.

Potential use for offensive stakeholders: identify high expected-points-per-shot areas and action types, and review low-probability shot situations where a better opportunity may be available. Potential use for defensive stakeholders: inspect difficult zones and action patterns where predicted make probability is lower. These are analytical decision-support ideas, not causal recommendations.

The model does not include defender distance, defensive pressure, shot-clock value, fatigue, score margin or player-tracking information. Strategy recommendations should therefore be interpreted within the limits of the recorded features and the Kobe Bryant historical sample.

10. Challenges and Mitigation

Missing target labels

Unlabelled rows cannot be used for supervised learning or accuracy evaluation. The notebook separates them and predicts their outcomes after training.

Evaluation leakage from random splits

Shots within the same game share context. Group-aware folds and a chronological hold-out are used to better approximate prediction on unseen future games.

Stacking leakage

Nested group-aware cross-fitting is used so the meta-model does not train on base predictions generated by models that have seen its validation fold.

Hold-out contamination

Model, calibration and threshold choices are made on development data. The chronological hold-out is reserved for final reporting.

Potentially valid outliers

Unusual long shots are inspected and retained when they are plausible basketball events.

Limited predictor context

The dataset lacks defender distance, shot-clock, fatigue and score-margin information. These omissions limit the predictive ceiling and interpretation.

Player-specific scope

The model is trained on Kobe Bryant's shots and requires new validation before application to other players.

11. Limitations

The model is trained on one player's historical shot records and should not be generalised to other players, teams or eras without retraining and evaluation. Shot outcomes are inherently noisy and the dataset omits several important contextual variables, including defender proximity, shot-clock state, fatigue and score margin.

Observed associations do not establish causation. SHAP and permutation importance describe how the fitted model uses its inputs; they do not prove that changing a feature would change the shot outcome. The unlabelled rows have no ground truth in the supplied dataset, so their predictions cannot be evaluated for accuracy from this file.

The supplied notebook has no executed outputs. Consequently, this report cannot state the observed dataset dimensions, final selected model, numerical performance, statistical significance, drift severity or highest/lowest efficiency zones as established results.

12. Future Enhancements

Integrate player-tracking data such as defender distance, defensive contest, player speed and positioning. Add shot-clock state, score margin, fatigue proxies and possession context when reliable data becomes available. Validate the approach across multiple players and seasons, using player-aware and time-aware evaluation.

Deploy the generated API and dashboard in a controlled environment, containerise the service, add input monitoring and define retraining triggers. Establish operational thresholds for PSI and model performance, and monitor calibration as well as classification metrics. Any deployment should include versioning, request validation, logging, privacy controls and periodic review.

13. Conclusion

The notebook defines a comprehensive end-to-end workflow for NBA shot analysis and Kobe Bryant shot-outcome prediction. It combines data validation, exploratory and statistical analysis, spatial and temporal feature engineering, grouped and chronological validation, comparison of seven classifiers, hyperparameter optimisation, ensembles, calibration, explainability, error analysis, drift monitoring and production-oriented artifact generation.

The central methodological strength is its explicit attention to leakage: games are kept together during grouped validation, later games are reserved for chronological hold-out evaluation, and model-selection decisions are made on development data. The project can support historical shot-efficiency analysis and probability-based decision support, subject to the limitations of the available variables and the single-player dataset.

Before final submission, execute the notebook from the first cell to the last, retain all outputs, and update the results section below with the computed values. Do not report unexecuted metrics as achieved results.

14. Results to Complete After Running the Notebook

Complete this table from the actual notebook outputs.

No.	Item	Value to enter
1	Dataset dimensions	30,697 rows × 25 columns
2	Labelled / unlabelled rows	25,697 / 5,000
3	Chronological split dates	Development: 1996-11-03 to 2011-01-04; Hold-out: 2011-01-05 to 2016-04-13
4	Selected model	HistGradientBoosting
5	Calibration method / threshold	No calibration / 0.385
6	Hold-out accuracy / F1	61.39% / 60.21%
7	Hold-out ROC-AUC / log-loss	0.6947 / 0.6176
8	Brier score	0.2146
9	Bootstrap confidence intervals	Accuracy: 0.5993–0.6306; F1: 0.5847–0.6190; ROC-AUC: 0.6786–0.7113
10	Important SHAP features	action_type, combined_shot_type, shot_distance, loc_y, opponent
11	Highest-efficiency zone	Restricted Area – 61.8% field-goal percentage
12	Drift status	Major drift: season_num, PSI 8.0684

Appendix A – Execution and Submission Checklist

- Open the notebook in the intended environment (Colab or Jupyter) and install dependencies.
- Run all cells from top to bottom on the original dataset.
- Resolve any validation failures or runtime errors and inspect warnings.
- Confirm that the final hold-out evaluation and model comparison tables are produced.
- Confirm that the model artifact, unlabelled predictions and generated service files are saved.
- Review the stakeholder insights and drift outputs; record actual values in Section 14.
- Save the executed notebook with outputs and submit it with this report and any required supporting artifacts.